Synchronization

DESCRIPTION

Many amusement parks in the US have bumper car rides. Each car holds one person. People who want to ride a bumper car line up and wait for a free car. When a car is returned and the previous rider gets out, a new rider gets in the car from the head of the waiting line and rides around on the bumper ride floor, bumping into other cars.

After riding around for a while, the rider in the bumper car decides that enough is enough and returns the car to the area where new riders line up. A new rider gets in and takes a ride in the bumper car. The rider who just got out of a bumper car wander around the amusement park, perhaps getting something to eat.

After wandering for a while, the rider decides to take another ride, returns to the rider waiting area, and gets in line. If there are more free cars than people wishing to ride, the free cars are kept in a line. A person wishing to ride a bumper car gets in the car at the head of this car-line and returning cars go to the end of this car-line.

There are N-riders rider threads (default 10) and N-cars bumper-car threads (default 5). Each car takes a different random amount of time between 0 and T-bump seconds (default 5) to bump around on the floor (simulated with the nap method) each time a rider gets into the car. The bump time (T-bump) is a random number re-generated each time a car goes onto the bumper floor. After completing a ride, each rider wanders around the amusement park (nap) for a random amount of time between 0 and T-wander seconds (default 10) before returning to the waiting area to get in line for another bumper car ride. The wander time, T-wander, is a random number re-generated each time a rider leaves a car and goes into the amusement park.

The simulation is run for T-total seconds (default 40). Each of the N-riders riders is simulated with an object containing a thread, created from a common class. Each bumper car is simulated with an object containing one thread, created from a common class. Finally, there is a driver class whose main method starts everything going.

GOALS

1. Learn multithread programming.
2. Learn Synchronization programming using Java monitor.

REQUIREMENTS

0: Semaphores are NOT allowed.

1: Below is pseudocode for each class. - Ask in class if you do NOT understand.

a. Coordinator.java:

```
class Coordinator extends Thread
```

```
            private boolean[] rideBegin, rideOver;
            private int car_riders;
            private boolean finish_loading;
            private int bump ;
            public  Coordinator( int riders, int cars, int
bumptime ) . . .
                  - initialize most data to zero
                  - create buffer as an array of ints with
size == cars
                  - create bool arrays, rideBegin and
rideOver size == riders
                  - initialize bool arrays to all false.
      /* the following are public synchronized methods */
            void getInLine( int rid )
                  - put rid into the buffer
                  - set rideBegin[rid] to false
                  - wait while rideBegin[rid] is false
            int load( )
                  - Get curRider from the buffer
                  - set rideBegin[curRider] to true
                  - signal
                  - set finish_loading to false
                  - wait while finish_loading is false
                  - return curRider

            void takeAseat( )
                  - set finish_loading to true
                  - signal
                  - increment car_riders
```

```
            void takeAride( int rid )
                  - set rideOver[rid] to false
                  - wait while rideOver[rid] is false


            void unload( int rid )
                  - set rideOver[rid] to true
                  - signal


      /* the following are public helper methods that are
NOT synchronized */
            int getBumpTime() -- returns amount of time a car
bumps
            int findTotalRider() -- returns the total number
of unloaded cars
```

Special Notes:

i. Copy and paste the private data from BBuffer.java and put them as private data of Coordinator.java.
ii. Copy and paste the public methods from BBuffer.java and put them as private methods in Coordinator.java.
iii. You MUST NOT have any prints and/or naps in any methods of class Coordinator.
iv. We will allow simultaneous loading. So even though the output indicates cars are "in line to be loaded" there will NOT be any line for the cars.
v. Make sure that any "waiting" is done in a while loop controlled by a boolean.
vi. Make sure to signal via notifyAll.

b. Rider.java:

```
class Rider implements Runnable
       private Coordinator coord = null;
       private int rid = -1; // Rider id #
       private boolean stop;
       public Rider(int id, Coordinator c) // Ask if you
don't know.
```

```
        {
            …
            new Thread(this).start();
        }
    public void run()
      while ( !stop )
        0. Generate random wander - time & write out
    wandering message (see sample output)
        1. nap  //   wander around the park
        2. write out that Rider is joining the line. (see
    sample output)
        3. coord.getInLine(. . .)
        4. actually take a second to take a seat :  nap 1000
        5. coord.takeAseat(. . .)
        6. coord.takeAride(. . .)
        7. write out that rider has finished riding. (see
    sample output)
     public void stopIt()
```

c. Car.java:

```
      class Car implements Runnable
      private Coordinator coord = null;
      private int cid = -1; // Car id #
      private boolean stop;
      public Car( int id, Coordinator c ) // Ask if you don't
know.
        {
    …
    new Thread(this).start();
        }
```

```
public void run()
   while ( !stop )
     0. Write out the Car + id "is in line to be loaded."
     1. rid = coord.load(. . .)   // car waits until loaded
     2. get bumpingTime
     3. bump (nap for bumpingTime) // bump on the floor
     4. actually take a second to unload :  nap 1000
     5. coord.unload(. . .)  // car waits until loaded
     6. write out Car + id + " has unloaded." (see sample
output)
   public void stopIt()
```

d. Bump.java:

```
public Bump
    public static void main(String[] args )
     1.  Create coordinator object
     2.  Create array of cars
     3.  Create array of riders
     4.  Create the cars
     5.  Create the riders
     6.  Write "All Car & Rider threads started."
     7.  nap for T-total seconds
     8.  Write "Calling stopIts; wait 15 seconds; do mini-
report"
     9.  Call the stopIt methods and wait 15 seconds.
    10.  Print mini-report
    11.  System.exit( 0 );
```

2: Do NOT make any changes to the files given:

GetOpt.java & SmallScheduler.java

3: Car.java & Rider.java: follow the pseudocode above.

Have a stopIt method to change the loop-control variable and thus exit the loop.

PAY ATTENTION TO THE SAMPLE OUTPUT in output.txt.

4: Follow the sample output.

5: Turn in the script file, part1, generated via the following commands.

```
javac *.java
cat Bump.java Car.java Coordinator.java Rider.java
java Bump -U
java Bump -r6 -c3 -b5 -w10 -R30
exit
```

6: Turn in the script file, part2, generated via the following commands.

```
javac *.java
java Bump
exit
```

SUBMISSION

1. Zipped source code (or screenshot of it).
2. Running result of script files, part1 and part2.

Rubric

Criteria	Ratings		Pts
Execute: Compile and execute correctly with no errors	**10 pts** Correct	**0 pts** Not correct or no submission	10 pts
Script: part1: The result is correct.	**5 pts** Correct	**0 pts** Not correct or no submission	5 pts
Script: part2: The result is correct.	**5 pts** Correct	**0 pts** Not correct or no submission	5 pts

Criteria	Ratings			Pts
Coordinator.java	**20 pts** Correct	**15 pts** Some errors	**0 pts** Not correct or no submission	20 pts
Rider.java	**20 pts** Correct	**15 pts** Some errors	**0 pts** Not correct or no submission	20 pts
Car.java	**20 pts** Correct	**15 pts** Some errors	**0 pts** Not correct or no submission	20 pts
Bump.java	**20 pts** Correct	**15 pts** Some errors	**0 pts** Not correct or no submission	20 pts
Total Points: 100				

CONGRATULATIONS, YOU'VE COMPLETED PROGRAM 6!